Klaus Tschira Stiftung

GESELLSCHAFT FÜR INFORMATIK

de RSE GESELLSCHAFT FÜR FORSCHUNGSSOFTWARE

RSE Award

Evaluation und Gütekriterien von Forschungssoftware

Fabian Lochner

2026-11-11

Contents

1. Zu meiner Person
2. Theorie
   - 2.1) Eure Papervorträge
   - 2.2) Kategorisierung von Forschungssoftware
   - 2.3) Evaluation von Forschungssoftware
3. Praxis (resqui tool)

Zu meiner Person

Zu meiner Person

Fabian Lochner
M.Sc. Social Data Science (Kopenhagen)

Evaluation von Forschungssoftware

Gesellschaft für Informatik e.V., Berlin
Projekt RSE Award

Gute Praxis sichtbar machen und exzellente Forschungssoftware auszeichnen, gemeinsam mit de-RSE e.V., gefördert von der Klaus Tschira Stiftung

Evaluation | RSE-Forschung

E-Mail: fabian.lochner@gi.de

Entwicklung von Forschungssoftware

Universität Oldenburg, Digital Social Science
Wiss. MA, Projekt RobustifAI

Computational Social Science: Forschungssoftware an der Schnittstelle von

Textdaten + Knowledge Graphs/ Ontologien + LLM-Agenten

E-Mail: fabian.lochner@uol.de

Theorie

2.1) Eure Papervorträge (und Diskussion)

2.2) Kategorisierung von Forschungssoftware

2.3) Evaluation von Forschungssoftware

2.1) Eure Papervorträge

Title	Creator	Year
EVERSE Reference Framework	Chue Hong et al.	2026
LiCoEval: Evaluating LLMs on License Compliance in Code Generation	Xu et al.	2025
Foundational Competencies and Responsibilities of a Research Software Engineer: Current State and Suggestions for Future Directions	Goth et al.	2025
GI- und DE-RSE Muster-Leitlinie zur effizienten Entwicklung von Forschungssoftware	Czerniak et al.	2025
Towards a quality indicator for research data publications and research software publications – a vision from the Helmholtz Association	Zu Castell et al.	2024
Multi-Dimensional Categorization of Research Software with Examples	Hasselbring et al.	2024
Introducing the FAIR Principles for research software	Barker et al.	2022
From FAIR research data toward FAIR and open research software	Hasselbring et al.	2020
Good enough practices in scientific computing	Wilson et al.	2017
Ten simple rules for making research software more robust	Taschuk and Wilson	2017
Software citation principles	Smith et al.	2016
Best Practices for Scientific Computing	Wilson et al.	2014
Open Science: One Term, Five Schools of Thought	Fecher and Friesike	2014
everse.software/indicators/website/indicators.html		
Measuring What Matters in the Age of AI: The Research Software Metrics Landscape	Malviya-Thakur et al.	

2.2) Kategorisierung von Forschungssoftware



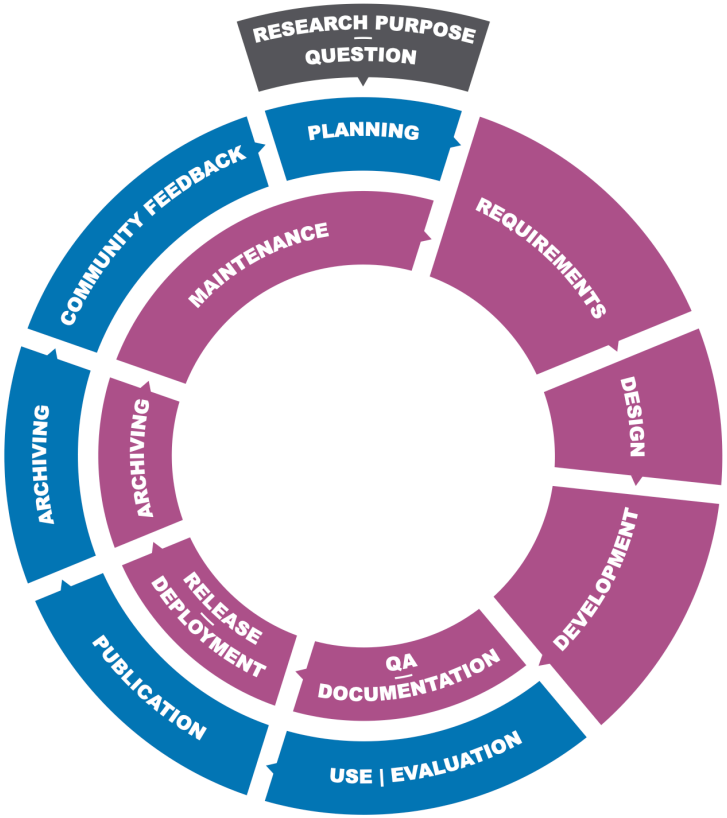
EVERSE 3-tier model

2.2.1) EVERSE 3-tier model

Tier	Definition	Use Cases	Features	Examples	Quality Implications
Analysis code	computational research processes and methodology	simulation, data generation, preparation, analysis and visualisation	personal use, scripts, small scope	Discrete representation in neural models	functionality, reproducability, basic documentation
Prototype tools	grew beyond the project in which it originated	answer multiple research questions	developed and used by multiple people	nipype: neuroimage python library	maintainability, testing, community interaction
Research software infrastructure	broadly applicable research software	bigger researcher community	large and possibly distributed development team	Jupyter Notebooks/Jupyter Lab, SciPy, NumPy, TensorFlow	security, scalability, sustainability

Quelle

2.2.2) Research Software Lifecycle



Quelle

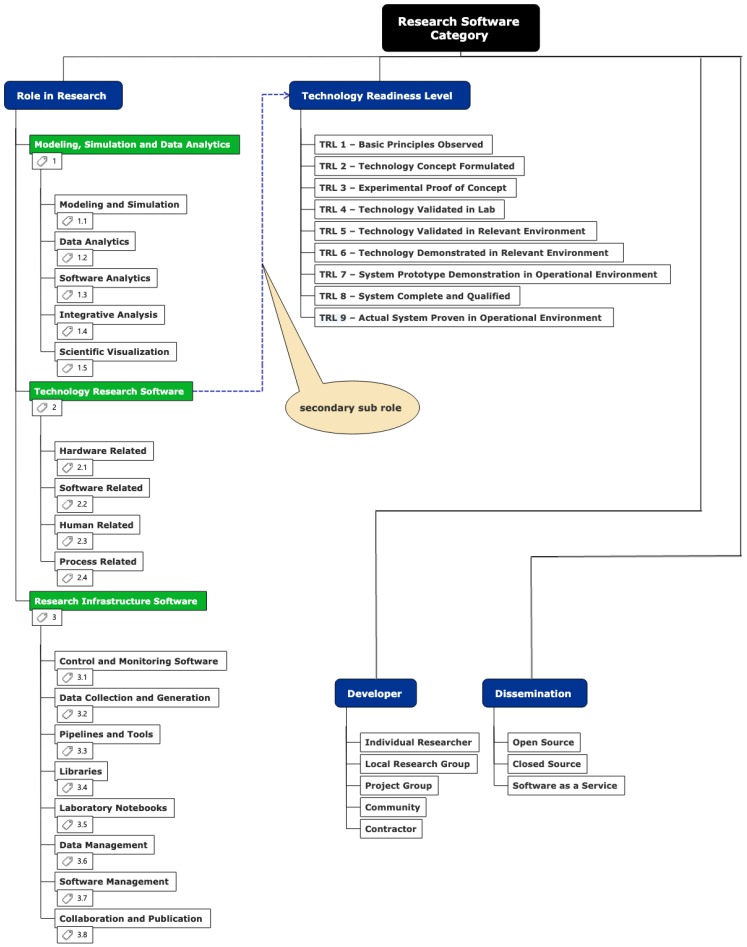
2.2.3) Quality Implications Across the Lifecycle

Different lifecycle stages emphasise different quality dimensions:

Lifecycle Phase	**Quality Focus**
Planning / Design	functional suitability, compatibility requirements
Development	maintainability, testability, security practices
Release / Deployment	reliability, performance efficiency, FAIR principles
Maintenance / Evolution	sustainability, community engagement

Quelle: EVERSE Reference Framework, Version 2.1

2.2.4) Kategorisierung nach Hasselbring et al. (2024)



2.3) Evaluation von Forschungssoftware

1. Welche Dimensionen und Kriterien für die Evaluation?
2. Wie vergleicht man Forschungssoftware aus verschiedenen Tiers und disziplinenübergreifend?
3. ...

2.3.1) EVERSE: Research Software Quality Dimensions and Indicators

Dimension	Description	Number of Indicators
FAIRness	Findable, Accessible, Interoperable, Reusable	21
Maintainability	Modularity, Reusability, Analysability, Modifiability, Testability	19
Security	Confidentiality, Integrity, Resistance etc.	5
Community	active communities of contributors, users, and stakeholders	4
Functional suitability	Functional completeness, correctness and appropriateness	4
Open Source Software	Open source software	4
Sustainability	The capacity of the software to endure	4
Flexibility	Adaptability, Scalability, Installability, Replaceability	2
Reliability	Faultlessness, Availability, Fault tolerance, Recoverability	2
Compatibility	Co-existence, Interoperability	1
Interaction Capability	Inclusivity, User assistance etc.	1
Safety	Risk identification, Hazard warning etc.	1
Performance Efficiency	Time behaviour, Resource utilization, Capacity	0

Quellen (abgerufen am 11.11.2026):

EVERSE: Research Software Quality Dimensions

EVERSE: Research Software Quality Indicators

2.3.2) Helmholtz Software Award

Award Categories: Best Scientific Software, Sustainability, Newcomer

Criteria	**Definition**	**Indicators**
Scientific Impact	Value of software to scientific research	Quality and number of journal citations
Use and Adoption within the Community	The extent to which the software is used and adopted by the scientific target community	references in papers, download statistics, GitHub stars
Software Engineering Level	High-quality engineering ensures maintainability, reproducibility, and reliability, long-term sustainability and ease of use	Adherence to best practices, development proccess, test automation, continuous integration etc.
Implementation of FAIR4RS and Open Science Principles	The FAIR4RS principles ensure that scientific software and its reference data can be efficiently used, cited, and maintained	Quality of metadata, continuity of release citations, re-use potential etc.
Young Software with High Growth Rate	Young but rapidly growing software may not yet have an extensive user base but shows strong momentum	Git statistics, user feedback, community interest etc.

2.3.3) RSE Award Prototype 1: Kategorien

Kategorie	Sub-Kategorie	Format	Teilnahmevoraussetzungen
Scientific Excellence	1.1 Geistes- und Sozialwissenschaften	Open Call	**1.** Die Forschungssoftware muss klar mit einer wissenschaftlichen, peer-reviewed Journal-Publikation verknüpft sein **2.** Mindestens ein*e Forscher*in/Entwickler*in des Teams muss mit einer Universität oder Forschungsinstitution im DACH-Raum verknüpft sein **3.** Nur bei 1.4: Die Forschungssoftware muss einen Software-Artefakt Badge einer wissenschaftlichen Konferenz haben
	1.2 Lebenswissenschaften		
	1.3 Natur- und Ingenieurwissenschaften		
	1.4 Informatik und Beiträge mit Software-Artefakt Badge		
	1.5 Interdisziplinäre Forschungssoftware/ Software Paper		
Newcomer		Open Call	**1.** PhD- und Masterstudierende einer DACH-Universität/Forschungsinstitution **2.** Einreichung eines Motivationsschreibens

2.3.4) RSE Award Prototype 1: Jury - Bewertungssystem

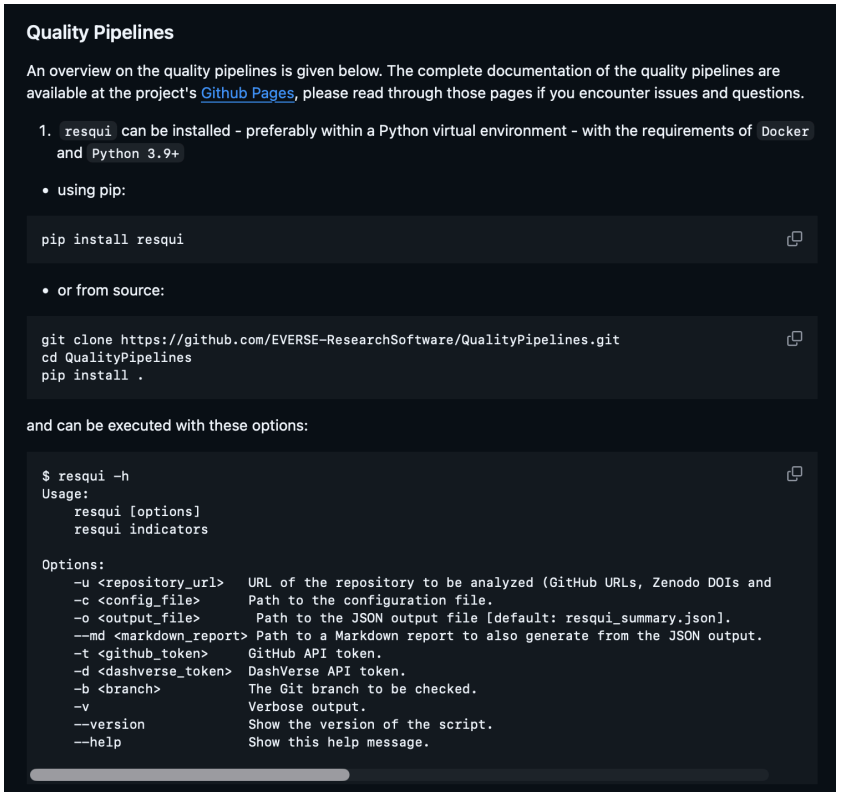
	Software Engineering Level	**Research Impact**	**Community Engagement**	**FAIRness & (Reproducibility)**	**Maintainability & Sustainability**
Definition	Dieses Kriterium bewertet die Einhaltung der Best Practices im Bereich des Software Engineerings	Dieses Kriterium bewertet die wissenschaftliche Relevanz	Dieses Kriterium bewertet den Grad des Engagements der Community für die Forschungssoftware	Dieses Kriterium bewertet die Einhaltung der FAIR4RS-Prinzipien	Dieses Kriterium bewertet die langfristige Stabilität und Wartbarkeit der Forschungssoftware
Indikatoren	Bewährte SE-Verfahren (z. B. Modularität, Lesbarkeit); Tests (Unit-, Integrations-, Systemtests); Testabdeckung; manuelle Codeüberprüfung (Pull-Requests)	Bewertung durch Fachgesellschaften	Repo-Beiträge (Mitwirkende, Pull-Anfragen, Commit-Häufigkeit, Issues); Repo-Beliebtheit (Stars, Forks, Downloads); Kommunikationskanäle und Dokumentation	CodeMeta-Vollständigkeit; Lizenz; permanenter Identifikator; archiviert in Software Heritage bzw. wissenschaftlichem Repositorium; Zitierangaben; containerisiert	Repo ist aktiv; Veröffentlichungen; aktuelle Metadaten; Abhängigkeitsverwaltung; Issue-Tracking-System; Phase im Software Development Life Cycle
Quellen	RSE Award Projektantrag, EVERSE indicators, Helmholtz Software Award				

Tabelle 2: Bewertungskriterien und Indikatoren. In der Kategorie **Scientific Excellence** gelten alle fünf Kriterien, in der Kategorie **Newcomer** nur *Software Engineering Level*, *Community Engagement* und *FAIRness*.

- Jedes Kriterium wird auf einer Ordinalskala von **1** (minimal) bis **5** (hervorragend) bewertet.
- In jeder (Sub-)Kategorie erhält das Projekt mit der höchsten Gesamtpunktzahl den **Scientific Excellence** bzw. **Newcomer Award**.

Praxis (resqui tool)

3.1) EVERSE Software Quality Pipelines: resqui tool



Quality Pipelines

An overview on the quality pipelines is given below. The complete documentation of the quality pipelines are available at the project's Github Pages, please read through those pages if you encounter issues and questions.

1. `resqui` can be installed - preferably within a Python virtual environment - with the requirements of `Docker` and `Python 3.9+`

- using pip:

```
pip install resqui
```

- or from source:

```
git clone https://github.com/EVERSE-ResearchSoftware/QualityPipelines.git
cd QualityPipelines
pip install .
```

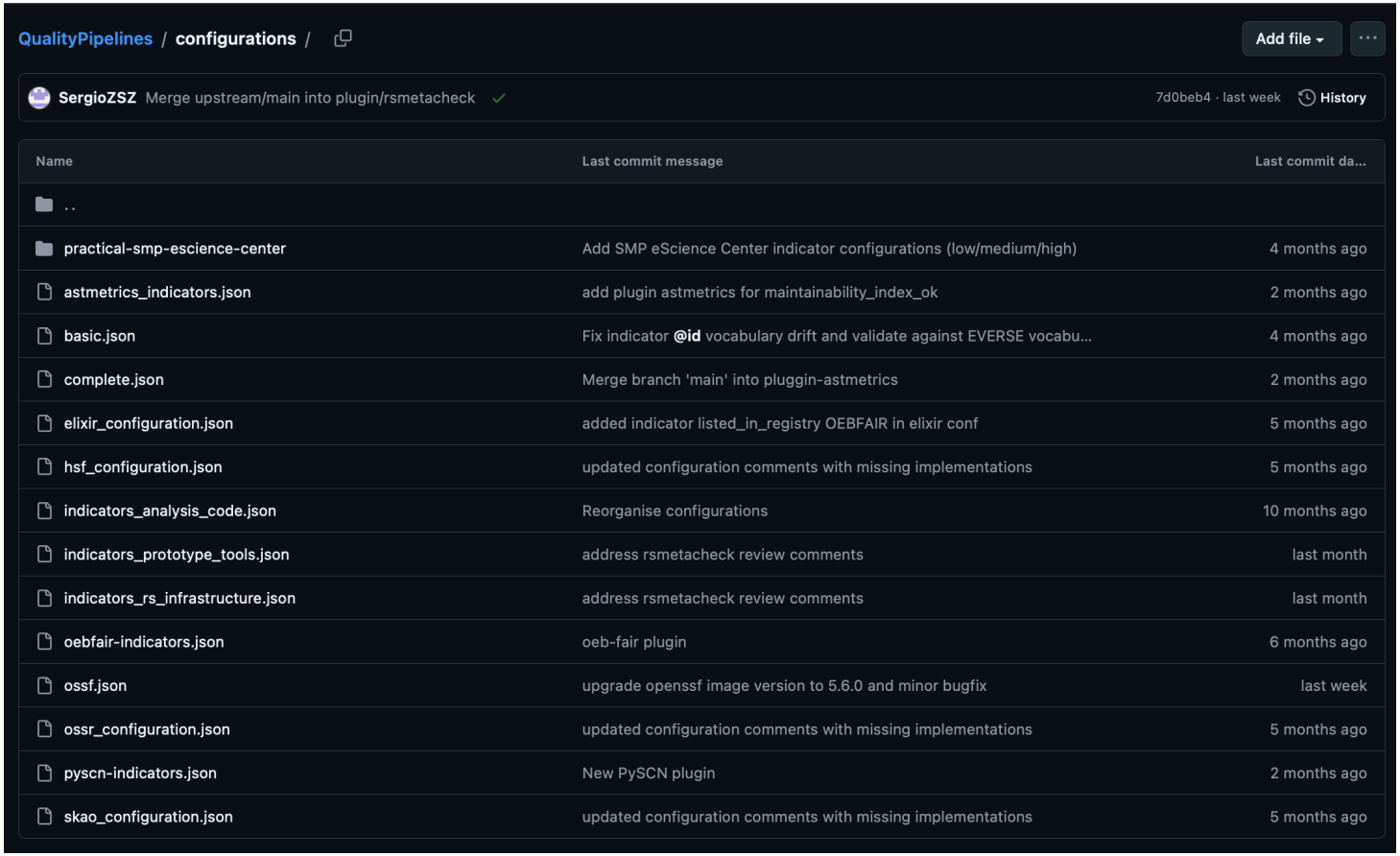
and can be executed with these options:

```
$ resqui -h
Usage:
    resqui [options]
    resqui indicators

Options:
    -u <repository_url>   URL of the repository to be analyzed (GitHub URLs, Zenodo DOIs and
    -c <config_file>      Path to the configuration file.
    -o <output_file>       Path to the JSON output file [default: resqui_summary.json].
    --md <markdown_report> Path to a Markdown report to also generate from the JSON output.
    -t <github_token>     GitHub API token.
    -d <dashverse_token>  DashVerse API token.
    -b <branch>           The Git branch to be checked.
    -v                    Verbose output.
    --version             Show the version of the script.
    --help                Show this help message.
```

Repository: github.com/EVERSE-ResearchSoftware/QualityPipelines
Dokumentation: everse.software/QualityPipelines

3.2) resqui: configuration files



QualityPipelines / configurations / Add file

SergioZSZ Merge upstream/main into plugin/rsmetacheck ✓ 7d0beb4 · last week History

Name	Last commit message	Last commit da...
..		
practical-smp-escience-center	Add SMP eScience Center indicator configurations (low/medium/high)	4 months ago
astmetrics_indicators.json	add plugin astmetrics for maintainability_index_ok	2 months ago
basic.json	Fix indicator **@id** vocabulary drift and validate against EVERSE vocabu...	4 months ago
complete.json	Merge branch 'main' into pluggin-astmetrics	2 months ago
elixir_configuration.json	added indicator listed_in_registry OEBFAIR in elixir conf	5 months ago
hsf_configuration.json	updated configuration comments with missing implementations	5 months ago
indicators_analysis_code.json	Reorganise configurations	10 months ago
indicators_prototype_tools.json	address rsmetacheck review comments	last month
indicators_rs_infrastructure.json	address rsmetacheck review comments	last month
oebfair-indicators.json	oeb-fair plugin	6 months ago
ossf.json	upgrade openssf image version to 5.6.0 and minor bugfix	last week
ossr_configuration.json	updated configuration comments with missing implementations	5 months ago
pyscn-indicators.json	New PySCN plugin	2 months ago
skao_configuration.json	updated configuration comments with missing implementations	5 months ago

https://github.com/EVERSE-ResearchSoftware/QualityPipelines/tree/main/configurations

3.3) resqui: Indicator coverage per dimension

Dimension	Number of Indicators	Implemented	Not implemented	Coverage
FAIRness	21	14	7	67%
Maintainability	19	11	8	58%
Security	5	5	0	100%
Community	4	1	3	25%
Functional suitability	4	1	3	25%
Open Source Software	4	1	3	25%
Sustainability	4	3	1	75%
Flexibility	2	2	0	100%
Reliability	2	1	1	50%
Compatibility	1	1	0	100%
Interaction Capability	1	1	0	100%
Safety	1	1	0	100%
Performance Efficiency	0	0	0	n/a
Sum of dimension tags	**68**	**42**	**26**	**62%**

Note: Some indicators are mapped to more than one dimension.

Status: 11.11.2026, resqui: complete.json

3.4) resqui: Not (yet) implemented indicators

Dimension	Count	Indicators
Maintainability	8	code_churn_ok, code_smells_ok, lines_of_code_ok, code_documentation_coverage_ok, uses_tool_for_warnings_and_mistakes, metadata_is_up_to_date, has_active_communication_channels, support_issue_tracking
FAIRness	7	archived_in_scholarly_repository, codemeta_completeness, software_has_license_for_file_types, metadata_is_up_to_date, passed_tests_ok, code_documentation_coverage_ok, uses_tool_for_warnings_and_mistakes
Community	3	has_active_communication_channels, has_active_contributors, response_timeframe_ok
Open Source Software	3	has_active_communication_channels, has_active_contributors, response_timeframe_ok
Functional suitability	3	passed_tests_ok, functional_correctness, software_test_coverage
Reliability	1	software_test_coverage
Sustainability	1	software_test_coverage

Note: An indicator can be mapped to more than one dimension. Status: 11.11.2026

3.5) resqui: initial test (2026-09-09) - quantiative results

Config	Repo	Passed indicators
complete_deduplicated_nolint.json	kg-digital-humanities	14/32
complete_deduplicated_nolint.json	lni_study	10/32
complete_deduplicated_nolint.json	graphrag	19/32
complete_deduplicated_nolint.json	FastSurfer	17/32
complete_deduplicated_nolint.json	kg-gen	13/32

3.6) resqui: initial test (2026-09-09) - qualitative results

Theme	Finding	Example
1) Consistency	Same indicator, different plugins: contradicting results. Evidence quality varies (RSFC detailed, OEBFAIR only “Found …”)	• descriptive_metadata: RSFC false vs. OEBFAIR true (correct one unclear) • persistent_and_unique_identifier: RSFC false vs. OEBFAIR true (RSFC correct) • has_package: OpenSSF Scorecard false vs. OEBFAIR true (OpenSSF correct) • Consistent: has_documentation, has_license, has_citation
2) Validity	Proxy checks (file and folder names) instead of content analysis; only GitHub and GitLab	• software_has_tests = true, only because of a notebook name and an empty “test” folder • has_contribution_guidelines: probably only checks that a file exists • version_control_use: Codeberg etc. not considered
3) Strictness	All sub-checks must pass, some never succeed: false negatives	descriptive_metadata = false for all four repos (“Languages”, “Date Created” never found, although present in FastSurfer’s pyproject.toml)
4) Verifiability	Verdicts hard to verify; identical results across repos	Gitleaks “insecure” hard to verify; no_critical_vulnerability = false for all four repos

Analysis based on own repo’s (kg-digital-humanities) results